11
Dynamic Programming

As algorithm designers and programmers, we are often charged with building a program to find the *best* solution for all problem instances. It is usually easy to write a program which gives a decent and correct solution, but ensuring that it always returns the absolute best solution requires us to think deeply about the problem.

Dynamic programming is a very powerful, general tool for solving optimization problems on left-right-ordered items such as character strings. Once understood it is relatively easy to apply, but many people have a difficult time understanding it.

Dynamic programming looks like magic until you have seen enough examples. Start by reviewing our binomial coefficient function in Chapter 6, as an example of how we stored partial results to help us compute what we were looking for. Then review Floyd's all-pairs shortest-path algorithm in Section 10.3.2. Only then should you study the two problems in the sections below. The first is a classic example of dynamic programming which appears in every textbook. The second is a more ad hoc example representative of using dynamic programming to design new algorithms.

11.1 Don't Be Greedy

Many problems call for finding the best solution satisfying certain constraints. We have a few tricks available to tackle such jobs. For example, the backtracking problems of Chapter 8 often asked for the largest, smallest, or highest-scoring configuration. Backtracking searches all possible solutions and selects the best one, and hence *must* return the correct answer. But this approach is only feasible for small problem instances.

Correct and efficient algorithms are known for many important graph problems, including shortest paths, minimum spanning trees, and matchings, as discussed in Chapter 10. Always be on the lookout for instances of these problems, so you can just plug in the appropriate solution.

Greedy algorithms focus on making the best local choice at each decision point. For example, a natural way to compute a shortest path from x to y might be to walk out of x, repeatedly following the cheapest edge until we get to y. Natural, but wrong! Indeed, in the absence of a correctness proof such greedy algorithms are very likely to fail.

So what can we do? Dynamic programming gives us a way to design custom algorithms which systematically search all possibilities (thus guaranteeing correctness) while storing results to avoid recomputing (thus providing efficiency).

Dynamic programming algorithms are defined by recursive algorithms/functions that describe the solution to the entire problem in terms of solutions to smaller problems. Backtracking is one such recursive procedure we have seen, as is depth-first search in graphs.

Efficiency in any such recursive algorithm requires storing enough information to avoid repeating computations we have done before. Why is depth-first search in graphs efficient? It is because we mark the vertices we have visited so we don't visit them again. Why is raw backtracking computationally expensive? Because it searches all possible paths/solutions instead of just the ones we haven't seen before.

Dynamic programming is a technique for efficiently implementing a recursive algorithm by storing partial results. The trick is to see that the naive recursive algorithm repeatedly computes the same subproblems over and over and over again. If so, storing the answers to them in a table instead of recomputing can lead to an efficient algorithm. To understand the examples which follow, it will help first to hunt for some kind of recursive algorithm. Only once you have a correct algorithm can you worry about speeding it up by using a results matrix.

11.2 Edit Distance

The problem of searching for patterns in text strings is of unquestionable importance. Indeed, we presented algorithms for string search in Chapter 3. However, there we limited discussion to *exact* string matching, finding where the pattern string s was exactly contained in the text string t. Life is often not that simple. Misspellings in either the text or pattern rob of us exact similarity. Evolutionary changes in genomic sequences or language usage imply that we often search with archaic patterns in mind: "Thou shalt not kill" morphs into "You should not murder."

If we are to deal with inexact string matching, we must first define a cost function telling us how far apart two strings are, i.e., a distance measure between pairs of strings. A reasonable distance measure minimizes the cost of the *changes* which have to be made to convert one string to another. There are three natural types of changes:

- *Substitution* — Change a single character from pattern s to a different character in text t, such as changing "shot" to "spot".

- *Insertion* — Insert a single character into pattern s to help it match text t, such as changing "ago" to "agog".
- *Deletion* — Delete a single character from pattern s to help it match text t, such as changing "hour" to "our".

Properly posing the question of string similarity requires us to set the cost of each of these string transform operations. Setting each operation to cost one step defines the *edit distance* between two strings. Other cost values also yield interesting results, as will be shown in Section 11.4.

But how can we compute the edit distance? We can define a recursive algorithm using the observation that the last character in the string must either be matched, substituted, inserted, or deleted. Chopping off the characters involved in the last edit operation leaves a pair of smaller strings. Let i and j be the last character of the relevant prefix of s and t, respectively. There are three pairs of shorter strings after the last operation, corresponding to the strings after a match/substitution, insertion, or deletion. *If* we knew the cost of editing the three pairs of smaller strings, we could decide which option leads to the best solution and choose that option accordingly. We *can* learn this cost, through the magic of recursion:

```
#define MATCH           0       /* enumerated type symbol for match */
#define INSERT          1       /* enumerated type symbol for insert */
#define DELETE          2       /* enumerated type symbol for delete */

int string_compare(char *s, char *t, int i, int j)
{
        int k;                  /* counter */
        int opt[3];             /* cost of the three options */
        int lowest_cost;        /* lowest cost */

        if (i == 0) return(j * indel(' '));
        if (j == 0) return(i * indel(' '));

        opt[MATCH] = string_compare(s,t,i-1,j-1) + match(s[i],t[j]);
        opt[INSERT] = string_compare(s,t,i,j-1) + indel(t[j]);
        opt[DELETE] = string_compare(s,t,i-1,j) + indel(s[i]);

        lowest_cost = opt[MATCH];
        for (k=INSERT; k<=DELETE; k++)
                if (opt[k] < lowest_cost) lowest_cost = opt[k];

        return( lowest_cost );
}
```

This program is absolutely correct – convince yourself. It is also impossibly slow. Running on our computer, it takes several seconds to compare two 11-character strings, and the computation disappears into never-never land on anything longer.

Why is the algorithm so slow? It takes exponential time because it recomputes values again and again and again. At every position in the string, the recursion branches three ways, meaning it grows at a rate of at least 3^n – indeed, even faster since most of the calls reduce only one of the two indices, not both of them.

So how can we make the algorithm practical? The important observation is that most of these recursive calls are computing things that have already been computed before. How do we know? Well, there can only be $|s| \cdot |t|$ possible unique recursive calls, since there are only that many distinct (i, j) pairs to serve as the parameters of recursive calls. By storing the values for each of these (i, j) pairs in a table, we can avoid recomputing them and just look them up as needed.

A table-based, dynamic programming implementation of this algorithm is given below. The table is a two-dimensional matrix m where each of the $|s| \cdot |t|$ cells contains the cost of the optimal solution of this subproblem, as well as a parent pointer explaining how we got to this location:

```
typedef struct {
        int cost;               /* cost of reaching this cell */
        int parent;             /* parent cell */
} cell;

cell m[MAXLEN+1][MAXLEN+1];     /* dynamic programming table */
```

The dynamic programming version has three differences from the recursive version. First, it gets its intermediate values using table lookup instead of recursive calls. Second, it updates the `parent` field of each cell, which will enable us to reconstruct the edit-sequence later. Third, it is instrumented using a more general `goal_cell()` function instead of just returning `m[|s|][|t|].cost`. This will enable us to apply this routine to a wider class of problems.

Be aware that we adhere to certain unusual string and index conventions in the following routines. In particular, we assume that each string has been padded with an initial blank character, so the first real character of string `s` sits in `s[1]`. This was done using the following input fragment:

```
s[0] = t[0] = ' ';
scanf("%s",&(s[1]));
scanf("%s",&(t[1]));
```

Why did we do this? It enables us to keep the matrix `m` indices in sync with those of the strings for clarity. Recall that we must dedicate the zeroth row and columns of `m` to store the boundary values matching the empty prefix. Alternatively, we could have left the input strings intact and just adjusted the indices accordingly.

```
int string_compare(char *s, char *t)
{
        int i,j,k;              /* counters */
        int opt[3];             /* cost of the three options */
```

```
        for (i=0; i<MAXLEN; i++) {
                row_init(i);
                column_init(i);
        }

        for (i=1; i<strlen(s); i++)
                for (j=1; j<strlen(t); j++) {
                        opt[MATCH] = m[i-1][j-1].cost + match(s[i],t[j]);
                        opt[INSERT] = m[i][j-1].cost + indel(t[j]);
                        opt[DELETE] = m[i-1][j].cost + indel(s[i]);

                        m[i][j].cost = opt[MATCH];
                        m[i][j].parent = MATCH;
                        for (k=INSERT; k<=DELETE; k++)
                                if (opt[k] < m[i][j].cost) {
                                        m[i][j].cost = opt[k];
                                        m[i][j].parent = k;
                                }
                }

        goal_cell(s,t,&i,&j);
        return( m[i][j].cost );
}
```

Notice the order that the cells of the matrix are filled in. To determine the value of cell (i, j) we need three values sitting and waiting for us, namely, the cells $(i-1, j-1)$, $(i, j-1)$, and $(i-1, j)$. Any evaluation order with this property will do, including the row-major order used in this program.

Below is an example run, showing the cost and parent values turning "thou shalt not" to "you should not" in five moves:

		y	o	u	-	s	h	o	u	l	d	-	n	o	t
:	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
t:	1	1	2	3	4	5	6	7	8	9	10	11	12	13	13
h:	2	2	2	3	4	5	5	6	7	8	9	10	11	12	13
o:	3	3	2	3	4	5	6	5	6	7	8	9	10	11	12
u:	4	4	3	2	3	4	5	6	5	6	7	8	9	10	11
-:	5	5	4	3	2	3	4	5	6	6	7	7	8	9	10
s:	6	6	5	4	3	2	3	4	5	6	7	8	8	9	10
h:	7	7	6	5	4	3	2	3	4	5	6	7	8	9	10
a:	8	8	7	6	5	4	3	3	4	5	6	7	8	9	10
l:	9	9	8	7	6	5	4	4	4	4	5	6	7	8	9
t:	10	10	9	8	7	6	5	5	5	5	5	6	7	8	8
-:	11	11	10	9	8	7	6	6	6	6	6	5	6	7	8
n:	12	12	11	10	9	8	7	7	7	7	7	6	5	6	7
o:	13	13	12	11	10	9	8	7	8	8	8	7	6	5	6
t:	14	14	13	12	11	10	9	8	8	9	9	8	7	6	5

		y	o	u	-	s	h	o	u	l	d	-	n	o	t
:	-1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
t:	2	0	0	0	0	0	0	0	0	0	0	0	0	0	0
h:	2	0	0	0	0	0	0	0	1	1	1	1	1	1	1
o:	2	0	0	0	0	0	0	0	1	1	1	1	1	0	1
u:	2	0	2	0	1	1	1	1	0	1	1	1	1	1	1
-:	2	0	2	2	0	1	1	1	1	0	0	0	1	1	1
s:	2	0	2	2	2	0	1	1	1	1	0	0	0	0	0
h:	2	0	2	2	2	2	0	1	1	1	1	1	1	0	0
a:	2	0	2	2	2	2	2	0	0	0	0	0	0	0	0
l:	2	0	2	2	2	2	2	0	0	0	1	1	1	1	1
t:	2	0	2	2	2	2	2	0	0	0	0	0	0	0	0
-:	2	0	2	2	0	2	2	0	0	0	0	0	1	1	1
n:	2	0	2	2	2	2	2	0	0	0	0	2	0	1	1
o:	2	0	0	2	2	2	2	0	0	0	0	2	2	0	1
t:	2	0	2	2	2	2	2	2	0	0	0	2	2	2	0

11.3 Reconstructing the Path

The dynamic programming implementation above returns the cost of the optimal solution, but not the solution itself. Knowing you can convert "thou shalt not" to "you should not" in five moves is fine and dandy, but what is the sequence of editing operations that does it?

The possible solutions to a given dynamic programming problem are described by paths through the dynamic programming matrix, starting from the initial configuration (the pair of empty strings $(0, 0)$) down to the final goal state (the pair of full strings $(|s|, |t|)$). The key to building the solution is to reconstruct the decisions made at every step along the path that lead to the optimal cost. These decisions have been recorded in the `parent` field of each array cell.

Reconstructing these decisions is done by walking backward from the goal state, following the `parent` pointer to an earlier cell. We repeat this process until we arrive back at the initial cell. The `parent` field for `m[i,j]` tells us whether the transform at (i, j) was `MATCH`, `INSERT`, or `DELETE`. The edit sequence from "thou-shalt-not" to "you-should-not" is `DSMMMMMISMSMMMM` – meaning delete the first "t", replace the "h" with "y", match the next five characters before inserting an "o", replace "a" with "u", and finally replace the "t" with a "d".

Walking backward reconstructs the solution in reverse order. However, clever use of recursion can do the reversing for us:

```
reconstruct_path(char *s, char *t, int i, int j)
{
        if (m[i][j].parent == -1) return;

        if (m[i][j].parent == MATCH) {
                reconstruct_path(s,t,i-1,j-1);
                match_out(s, t, i, j);
                return;
        }
        if (m[i][j].parent == INSERT) {
                reconstruct_path(s,t,i,j-1);
                insert_out(t,j);
                return;
        }
        if (m[i][j].parent == DELETE) {
                reconstruct_path(s,t,i-1,j);
                delete_out(s,i);
                return;
        }
}
```

For many problems, including edit distance, the tour can be reconstructed without explicitly retaining the last-move pointer array. The trick is working backward to see what move took you to the current cell at the given cost.

11.4 Varieties of Edit Distance

The optimization and path reconstruction routines given above reference several functions which we have not yet defined. These fall into four categories:

- *Table Initialization* — The functions `row_init()` and `column_init()` initialize the zeroth row and column of the dynamic programming table, respectively. For the string edit distance problem, cells $(i, 0)$ and $(0, i)$ correspond to matching length-i strings against the empty string. This requires exactly i insertions/deletions, so the definition of these functions is clear.

```
row_init(int i)
{
  m[0][i].cost = i;
  if (i>0)
       m[0][i].parent =  INSERT;
  else
       m[0][i].parent = -1;
}
```

```
column_init(int i)
{
  m[i][0].cost = i;
  if (i>0)
       m[i][0].parent = DELETE;
  else
       m[0][i].parent = -1;
}
```

- *Penalty Costs* — The functions `match(c,d)` and `indel(c)` present the costs for transforming character c to d and inserting/deleting character c. For standard edit distance, `match` should cost nothing if the characters are identical, and 1 otherwise, while `indel` returns 1 regardless of what the argument is. But more sensitive cost functions are certainly possible, perhaps more forgiving of replacements located near each other on standard keyboard layouts or those which sound or look similar.

```
int match(char c, char d)
{
  if (c == d) return(0);
  else return(1);
}
```

```
int indel(char c)
{
  return(1);
}
```

- *Goal Cell Identification* — The function `goal_cell` returns the indices of the cell marking the endpoint of the solution. For edit distance, this is defined by the length of the two input strings. However, other applications we shall see do not have fixed goal locations.

```
goal_cell(char *s, char *t, int *i, int *j)
{
  *i = strlen(s) - 1;
  *j = strlen(t) - 1;
}
```

- *Traceback Actions* — The functions `match_out`, `insert_out`, and `delete_out` perform the appropriate actions for each edit-operation during traceback. For

edit distance, this might mean printing out the name of the operation or character involved, as determined by the needs of the application.

```
insert_out(char *t, int j)
{
        printf("I");
}

delete_out(char *s, int i)
{
        printf("D");
}
```

```
match_out(char *s, char *t,
          int i, int j)
{
    if (s[i]==t[j]) printf("M");
    else printf("S");
}
```

For our edit distance computation all of these functions are quite simple. However, we must confess about the difficulty of getting the boundary conditions and index manipulations correct. Although dynamic programming algorithms are easy to design once you understand the technique, getting the details right requires carefully thinking and thorough testing.

This is a lot of infrastructure to develop for such a simple algorithm. However, there are several important problems which can now be solved as special cases of edit distance using only minor changes to some of these stubs.

- *Substring Matching* — Suppose that we want to find where a short pattern s best occurs within a long text t, say, searching for "Skiena" in all its misspellings (Skienna, Skena, Skina, ...). Plugging this search into our original edit distance function will achieve little sensitivity, since the vast majority of any edit cost will be that of deleting the body of the text.

  We want an edit distance search where the cost of starting the match is independent of the position in the text, so that a match in the middle is not prejudiced against. Likewise, the goal state is not necessarily at the end of both strings, but the cheapest place to match the entire pattern somewhere in the text. Modifying these two functions gives us the correct solution:

```
row_init(int i)
{
        m[0][i].cost = 0;       /* note change */
        m[0][i].parent = -1;    /* note change */
}

goal_cell(char *s, char *t, int *i, int *j)
{
        int k;                  /* counter */

        *i = strlen(s) - 1;
        *j = 0;
        for (k=1; k<strlen(t); k++)
                if (m[*i][k].cost < m[*i][*j].cost) *j = k;
```

```
}
```

- *Longest Common Subsequence* — Often we are interested in finding the longest scattered string of characters which is included within both words. The *longest common subsequence* (LCS) between "democrat" and "republican" is *eca*.

  A common subsequence is defined by all the identical-character matches in an edit trace. To maximize the number of such traces, we must prevent substitution of non-identical characters. This is done by changing the match-cost function:

```
int match(char c, char d)
{
        if (c == d) return(0);
        else return(MAXLEN);
}
```

  Actually, it suffices to make the substitution penalty greater than that of an insertion plus a deletion for substitution to lose any allure as an edit operation.

- *Maximum Monotone Subsequence* — A numerical sequence is *monotonically increasing* if the ith element is at least as big as the $(i-1)$st element. The *maximum monotone subsequence* problem seeks to delete the fewest number of elements from an input string S to leave a monotonically increasing subsequence. Thus a longest increasing subsequence of "243517698" is "23568."

  In fact, this is just a longest common subsequence problem, where the second string is the elements of S sorted in increasing order. Any common sequence of these two must (a) represent characters in proper order in S, and (b) use only characters with increasing position in the collating sequence, so the longest one does the job. Of course, this approach can be modified to give the longest decreasing sequence by simply reversing the sorted order.

As you can see, our simple edit distance routine can be made to do many amazing things easily. The trick is observing that your problem is just a special case of approximate string matching.

11.5 Program Design Example: Elevator Optimization

I work in a very tall building with a very slow elevator. It is especially frustrating for me when people press the buttons for many neighboring floors (say 13, 14, and 15) as I go from ground to the top floor. My trip upward is interrupted three times, once at each of these floors. It would be far more polite for the three of them to agree to press only 14, and have the floor 13 and 15 people take the stairs up or down for one floor. They could use the walk, anyway.

Your task is to write an elevator optimization program. The riders all enter their intended destinations at the beginning of the trip. The elevator then decides which floors it will stop at along the way. We limit the elevator to making at most k stops on any given run, but select the floors so as to minimize the total number of floors people

have to walk either up or down. You can assume the elevator is smart enough to know how many people want to get off at each floor.

We assume that the penalty for walking up a flight of stairs is the same as walking down one – remember, these people can use the exercise. In this spirit, management proposes to break ties among equal-cost solutions by giving preference to stopping the elevators at the lowest floor possible, since it uses less electricity. Note that the elevator does not necessarily have to stop at one of the floors the riders specified. If riders specify floors 27 and 29, it can decide to stop at floor 28 instead.

Solution starts below

This is an example of a typical programming/algorithm problem which can be neatly solved by dynamic programming. How do we recognize that, and once we do how do we solve it?

Recall that dynamic programming algorithms are based on recursive algorithms. Deciding the best place to put the kth stop depends on the cost of all possible solutions with $k-1$ stops. If you can tell me the cost of the best of the relevant partial solutions, I can make the right decision for the final stop.

Efficient dynamic programming algorithms tend to require *ordered* input. The fact that the passenger's destinations can be ordered from lowest to highest is important. Consider a trip that stops at floor f_2, after an initial stop at f_1. This second stop can be of absolutely no interest to any passenger whose real destination is at or below f_1. This means that the problem can be decomposed into pieces. If I want to add a third stop f_3 above f_2, plotting the best location for it requires no knowledge of f_1.

So we smell dynamic programming here. What is the algorithm? We need to define a cost function for the partial solutions which will let us make bigger decisions. How about —

> Let `m[i][j]` denote the minimum cost of serving *all* the riders using exactly j stops, the last of which is at floor i.

Can this function help us place the $(j+1)$st stop given smaller values? Yes. The $(j+1)$st stop must, by definition, be higher than the previous (jth) stop at floor i. Further the new stop will only be of interest to the passengers seeking to get above the ith floor. To figure out how much it can help, we must properly divide the passengers between the new stop and i based on which stop they are closer to. This idea defines the following recurrence:

$$m_{i,j+1} = \min_{k=0}^{i}(m_{k,j} - \text{floors_walked}(k,\infty) + \text{floors_walked}(k,i) + \text{floors_walked}(i,\infty))$$

What does the recurrence mean? If the last stop is at i, the previous stop has to be at some $k < i$. What will such a solution cost? We must subtract from $m_{k,j}$ the cost of servicing all passengers above k (i.e., floors_walked(k,∞)), and replace it by the (presumably) reduced cost of adding a stop at i (i.e., floors_walked(k,i) + floors_walked(i,∞)).

The key is the function `floors_walked(a,b)`, which counts the total number of floors walked by passengers whose destinations are between the two consecutive stops a and b. Each such passenger goes to the closest such stop:

```
floors_walked(int previous, int current)
{
        int nsteps=0;           /* total distance traveled */
        int i;                  /* counter */

        for (i=1; i<=nriders; i++)
              if ((stops[i] > previous) && (stops[i] <= current))
                   nsteps += min(stops[i]-previous, current-stops[i]);

        return(nsteps);
}
```

Once you buy this logic, the implementation of this algorithm becomes straightforward. We set up global matrices to hold the dynamic programming tables, here separated to store the cost and parent fields:

```
#define NFLOORS         110     /* the building height in floors */
#define MAX_RIDERS      50      /* what is the elevator capacity? */

int stops[MAX_RIDERS];          /* what floor does everyone get off? */
int nriders;                    /* number of riders */
int nstops;                     /* number of allowable stops */

int m[NFLOORS+1][MAX_RIDERS];   /* dynamic programming cost table */
int p[NFLOORS+1][MAX_RIDERS];   /* dynamic programming parent table */
```

The optimization function is a direct implementation of the recurrence, with care taken to order the loops so that all values are ready before they are needed:

```
int optimize_floors()
{
        int i,j,k;              /* counters */
        int cost;               /* costs placeholder */
        int laststop;           /* the elevator's last stop */

        for (i=0; i<=NFLOORS; i++) {
                m[i][0] = floors_walked(0,MAXINT);
                p[i][0] = -1;
        }

        for (j=1; j<=nstops; j++)
              for (i=0; i<=NFLOORS; i++) {
                   m[i][j] = MAXINT;
```

```
                for (k=0; k<=i; k++) {
                     cost = m[k][j-1] - floors_walked(k,MAXINT) +
                        floors_walked(k,i) + floors_walked(i,MAXINT);
                     if (cost < m[i][j]) {
                        m[i][j] = cost;
                        p[i][j] = k;
                     }
                }
           }

        laststop = 0;
        for (i=1; i<=NFLOORS; i++)
                if (m[i][nstops] < m[laststop][nstops])
                     laststop = i;

        return(laststop);
}
```

Finally, we need to reconstruct the solution. The logic is exactly the same as the previous examples: follow the parent pointers and work backward:

```
reconstruct_path(int lastfloor, int stops_to_go)
{
        if (stops_to_go > 1)
             reconstruct_path(p[lastfloor][stops_to_go], stops_to_go-1);

        printf("%d\n",lastfloor);
}
```

Running this program on a ten-story European building (which has the ground floor labeled zero) with single passengers seeking to go to every floor from 1 to 10 informs us that the best single stop is at floor 7, for a cost of 18 walked flights (the floor 1, 2, and 3 passengers are told to get out and walk up from the ground floor). The best pair of stops are 3 and 8 for a cost of 11, while the best triple of stops is at 3, 6, and 9 for a total cost of 7 flights.

11.6 Problems

11.6.1 Is Bigger Smarter?

PC/UVa IDs: 111101/10131, **Popularity:** B, **Success rate:** high **Level:** 2

Some people think that the bigger an elephant is, the smarter it is. To disprove this, you want to analyze a collection of elephants and place as large a subset of elephants as possible into a sequence whose weights are increasing but IQ's are decreasing.

Input

The input will consist of data for a bunch of elephants, at one elephant per line terminated by the end-of-file. The data for each particular elephant will consist of a pair of integers: the first representing its size in kilograms and the second representing its IQ in hundredths of IQ points. Both integers are between 1 and 10,000. The data contains information on at most 1,000 elephants. Two elephants may have the same weight, the same IQ, or even the same weight and IQ.

Output

The first output line should contain an integer n, the length of elephant sequence found. The remaining n lines should each contain a single positive integer representing an elephant. Denote the numbers on the ith data line as $W[i]$ and $S[i]$. If these sequence of n elephants are $a[1]$, $a[2]$,..., $a[n]$ then it must be the case that

$$W[a[1]] < W[a[2]] < ... < W[a[n]] \text{ and } S[a[1]] > S[a[2]] > ... > S[a[n]]i$$

In order for the answer to be correct, n must be as large as possible. All inequalities are strict: weights must be strictly increasing, and IQs must be strictly decreasing.

Your program can report any correct answer for a given input.

Sample Input

```
6008 1300
6000 2100
500 2000
1000 4000
1100 3000
6000 2000
8000 1400
6000 1200
2000 1900
```

Sample Output

```
4
4
5
9
7
```

11.6.2 Distinct Subsequences

PC/UVa IDs: 111102/10069, **Popularity:** B, **Success rate:** average **Level:** 3

A subsequence of a given sequence S consists of S with zero or more elements deleted. Formally, a sequence $Z = z_1 z_2 \ldots z_k$ is a subsequence of $X = x_1 x_2 \ldots x_m$ if there exists a strictly increasing sequence $< i_1, i_2, \ldots, i_k >$ of indices of X such that for all $j = 1, 2, \ldots, k$, we have $x_{i_j} = z_j$. For example, $Z = bcdb$ is a subsequence of $X = abcbdab$ with corresponding index sequence $< 2, 3, 5, 7 >$.

Your job is to write a program that counts the number of occurrences of Z in X as a subsequence such that each has a distinct index sequence.

Input

The first line of the input contains an integer N indicating the number of test cases to follow. The first line of each test case contains a string X, composed entirely of lowercase alphabetic characters and having length no greater than 10,000. The second line contains another string Z having length no greater than 100 and also composed of only lowercase alphabetic characters. Be assured that neither Z nor any prefix or suffix of Z will have more than 10^{100} distinct occurrences in X as a subsequence.

Output

For each test case, output the number of distinct occurrences of Z in X as a subsequence.

Output for each input set must be on a separate line.

Sample Input

```
2
babgbag
bag
rabbbit
rabbit
```

Sample Output

```
5
3
```

11.6.3 Weights and Measures

PC/UVa IDs: 111103/10154, **Popularity:** C, **Success rate:** average **Level:** 3

A turtle named Mack, to avoid being cracked, has enlisted your advice as to the order in which turtles should be stacked to form Yertle the Turtle's throne. Each of the 5,607 turtles ordered by Yertle has a different weight and strength. Your task is to build the largest stack of turtles possible.

Input

Standard input consists of several lines, each containing a pair of integers separated by one or more space characters, specifying the weight and strength of a turtle. The weight of the turtle is in grams. The strength, also in grams, is the turtle's overall carrying capacity, including its own weight. That is, a turtle weighing 300 g with a strength of 1,000 g can carry 700 g of turtles on its back. There are at most 5,607 turtles.

Output

Your output is a single integer indicating the maximum number of turtles that can be stacked without exceeding the strength of any one.

Sample Input

```
300 1000
1000 1200
200 600
100 101
```

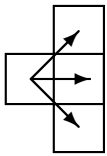
Sample Output

```
3
```

11.6.4 Unidirectional TSP

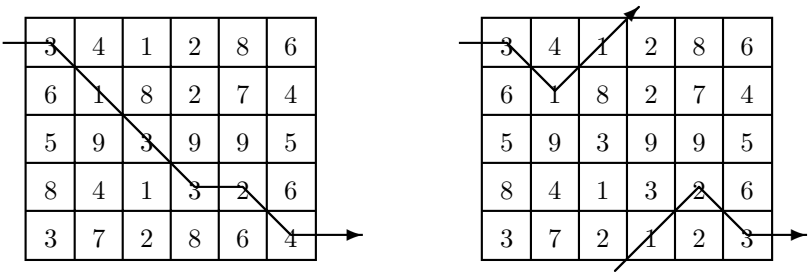
PC/UVa IDs: 111104/116, **Popularity:** A, **Success rate:** low **Level:** 3

Given an $m \times n$ matrix of integers, you are to write a program that computes a path of minimal weight from left to right across the matrix. A path starts anywhere in column 1 and consists of a sequence of steps terminating in column n. Each step consists of traveling from column i to column $i + 1$ in an adjacent (horizontal or diagonal) row. The first and last rows (rows 1 and m) of a matrix are considered adjacent; i.e., the matrix "wraps" so that it represents a horizontal cylinder. Legal steps are illustrated below.



The *weight* of a path is the sum of the integers in each of the n cells of the matrix that are visited.

The minimum paths through two slightly different 5×6 matrices are shown below. The matrix values differ only in the bottom row. The path for the matrix on the right takes advantage of the adjacency between the first and last rows.



Input

The input consists of a sequence of matrix specifications. Each matrix consists of the row and column dimensions on a line, denoted m and n, respectively. This is followed by $m \cdot n$ integers, appearing in row major order; i.e., the first n integers constitute the first row of the matrix, the second n integers constitute the second row, and so on. The integers on a line will be separated from other integers by one or more spaces. Note: integers are *not* restricted to being positive. There will be one or more matrix specifications in an input file. Input is terminated by end-of-file.

For each specification the number of rows will be between 1 and 10 inclusive; the number of columns will be between 1 and 100 inclusive. No path's weight will exceed integer values representable using 30 bits.

Output

Two lines should be output for each matrix specification. The first line represents a minimal-weight path, and the second line is the cost of this minimal path. The path consists of a sequence of n integers (separated by one or more spaces) representing the rows that constitute the minimal path. If there is more than one path of minimal weight, the lexicographically smallest path should be output.

Sample Input

```
5 6
3 4 1 2 8 6
6 1 8 2 7 4
5 9 3 9 9 5
8 4 1 3 2 6
3 7 2 8 6 4
5 6
3 4 1 2 8 6
6 1 8 2 7 4
5 9 3 9 9 5
8 4 1 3 2 6
3 7 2 1 2 3
2 2
9 10 9 10
```

Sample Output

```
1 2 3 4 4 5
16
1 2 1 5 4 5
11
1 1
19
```

11.6.5 Cutting Sticks

PC/UVa IDs: 111105/10003, **Popularity:** B, **Success rate:** average **Level:** 2

You have to cut a wood stick into several pieces. The most affordable company, Analog Cutting Machinery (ACM), charges money according to the length of the stick being cut. Their cutting saw allows them to make only one cut at a time.

It is easy to see that different cutting orders can lead to different prices. For example, consider a stick of length 10 m that has to be cut at 2, 4, and 7 m from one end. There are several choices. One can cut first at 2, then at 4, then at 7. This leads to a price of $10 + 8 + 6 = 24$ because the first stick was of 10 m, the resulting stick of 8 m, and the last one of 6 m. Another choice could cut at 4, then at 2, then at 7. This would lead to a price of $10 + 4 + 6 = 20$, which is better for us.

Your boss demands that you write a program to find the minimum possible cutting cost for any given stick.

Input

The input will consist of several input cases. The first line of each test case will contain a positive number l that represents the length of the stick to be cut. You can assume $l < 1{,}000$. The next line will contain the number n ($n < 50$) of cuts to be made.

The next line consists of n positive numbers c_i ($0 < c_i < l$) representing the places where the cuts must be made, given in strictly increasing order.

An input case with $l = 0$ represents the end of input.

Output

Print the cost of the minimum cost solution to cut each stick in the format shown below.

Sample Input

```
100
3
25 50 75
10
4
4 5 7 8
0
```

Sample Output

```
The minimum cutting is 200.
The minimum cutting is 22.
```

11.6.6 Ferry Loading

PC/UVa IDs: 111106/10261, **Popularity:** B, **Success rate:** low **Level:** 3

Ferries are used to transport cars across rivers and other bodies of water. Typically, ferries are wide enough to support two lanes of cars throughout their length. The two lanes of cars drive onto the ferry from one end, the ferry crosses the water, and the cars exit from the other end of the ferry.

The cars waiting to board the ferry form a single queue, and the operator directs each car in turn to drive onto the port (left) or starboard (right) lane of the ferry so as to balance the load. Each car in the queue has a different length, which the operator estimates by inspecting the queue. Based on this inspection, the operator decides which side of the ferry each car should board, and boards as many cars as possible from the queue, subject to the length limit of the ferry. Write a program that will tell the operator which car to load on which side so as to maximize the number of cars loaded.

Input

The input begins with a single positive integer on a line by itself indicating the number of test cases, followed by a blank line.

The first line of each test case contains a single integer between 1 and 100: the length of the ferry (in meters). For each car in the queue there is an additional line of input specifying the length of the car in cm, an integer between 100 and 3,000 inclusive. A final line of input contains the integer 0. The cars must be loaded in order, subject to the constraint that the total length of cars on either side does not exceed the length of the ferry. As many cars should be loaded as possible, starting with the first car in the queue and loading cars in order until the next car cannot be loaded.

There is a blank line between each two consecutive inputs.

Output

For each test case, the first line of output should give the number of cars that can be loaded onto the ferry. For each car that can be loaded onto the ferry, in the order the cars appear in the input, output a line containing "`port`" if the car is to be directed to the port side and "`starboard`" if the car is to be directed to the starboard side. If several arrangements of cars meet the criteria above, any one will do.

The output of two consecutive cases will be separated by a blank line.

Sample Input

```
1

50
2500
3000
```

```
1000
1000
1500
700
800
0
```

Sample Output

```
6
port
starboard
starboard
starboard
port
port
```

11.6.7 Chopsticks

PC/UVa IDs: 111107/10271, **Popularity:** B, **Success rate:** average **Level:** 3

In China, people use pairs of chopsticks to eat food, but Mr. L is a bit different. He uses a set of *three* chopsticks, one pair plus an extra; a long chopstick to get large items by stabbing the food. The length of the two shorter, standard chopsticks should be as close as possible, but the length of the extra one is not important so long as it is the longest. For a set of chopsticks with lengths A, B, C ($A \leq B \leq C$), the function $(A - B)^2$ defines the "*badness*" of the set.

Mr. L has invited K people to his birthday party, and he is eager to introduce his way of using chopsticks. He must prepare $K + 8$ sets of chopsticks (for himself, his wife, his little son, little daughter, his mother, father, mother-in-law, father-in-law, and K other guests). But Mr. L's chopsticks are of many different lengths! Given these lengths, he must find a way of composing the $K + 8$ sets such that the total badness of the sets is minimized.

Input

The first line in the input contains a single integer T indicating the number of test cases ($1 \leq T \leq 20$). Each test case begins with two integers K and N ($0 \leq K \leq 1{,}000$, $3K + 24 \leq N \leq 5{,}000$) giving the number of guests and the number of chopsticks. Then follow N positive integers L_i, in non–decreasing order, indicating the lengths of the chopsticks ($1 \leq L_i \leq 32{,}000$).

Output

For each test case in the input, print a line containing the minimal total badness of all the sets.

Sample Input

```
1
1 40
1 8 10 16 19 22 27 33 36 40 47 52 56 61 63 71 72 75 81 81 84 88 96 98
103 110 113 118 124 128 129 134 134 139 148 157 157 160 162 164
```

Sample Output

```
23
```

Note: A possible collection of the nine chopstick sets for this sample input is $(8, 10, 16)$, $(19, 22, 27)$, $(61, 63, 75)$, $(71, 72, 88)$, $(81, 81, 84)$, $(96, 98, 103)$, $(128, 129, 148)$, $(134, 134, 139)$, and $(157, 157, 160)$.

11.6.8 Adventures in Moving: Part IV

PC/UVa IDs: 111108/10201, **Popularity:** A, **Success rate:** low **Level:** 3

You are considering renting a moving truck to help you move from Waterloo to the big city. Gas prices being so high these days, you want to know how much the gas for this beast will set you back.

The truck consumes a full liter of gas for each kilometer it travels. It has a 200-liter gas tank. When you rent the truck in Waterloo, the tank is half-full. When you return it in the big city, the tank must be at least half-full, or you'll get gouged even more for gas by the rental company. You would like to spend as little as possible on gas, but you don't want to run out along the way.

Input

The input begins with a single positive integer on a line by itself indicating the number of test cases, followed by a blank line.

Each test case is composed only of integers. The first integer is the distance in kilometers from Waterloo to the big city, at most 10,000. Next comes a set of up to 100 gas station specifications, describing all the gas stations along your route, in non-decreasing order by distance. Each specification consists of the distance in kilometers of the gas station from Waterloo, and the price of a liter of gas at the gas station, in tenths of a cent, at most 2,000.

There is a blank line between each two consecutive inputs.

Output

For each test case, output the minimum amount of money that you can spend on gas to get from Waterloo to the big city. If it is not possible to get from Waterloo to the big city subject to the constraints above, print "`Impossible`".

The output of each two consecutive cases will be separated by a blank line.

Sample Input

```
1

500
100 999
150 888
200 777
300 999
400 1009
450 1019
500 1399
```

Sample Output

```
450550
```

11.7 Hints

11.1 Can this be reduced to some form of string matching problem?

11.3 Does the original order of the input have any meaning, or are we free to rearrange it? If so, what order is most useful?

11.4 What information do we need about shorter tours to be able to select the optimal last move?

11.5 Can we exploit the fact that each cut leaves two smaller sticks to construct a recursive algorithm?

11.6 Does always putting the next car on the side with the most remaining room solve the problem? Why or why not? Can we exploit the fact that the sum of accumulated car lengths on each lane of the ferry is always an integer?

11.7 How could we solve the problem if we didn't have to worry about the third chopstick?

11.8 What information about the costs of reaching certain places with certain amounts of gas is sufficient to select the optimal last move?

11.8 Notes

11.3 More about Yertle the Turtle can be found in [Seu58].